Introduction

If you have not done so already, please be sure to carefully read:

- overview
- description of the project structure

Phase 1 (including Phase 0) of the term project is worth 30% of your overall project grade.

Academic Honesty

All the code that you submit at each phase of the term project must be

- code that you have written yourself since the start of this term (you *must not* make use of *any* code that you have written in the past)
- code found in or based on any of the projects that we present to you in lecture during the term, provided you insert a comment into your code to attribute the source

Please ensure that you do not provide other students with access to your code. By doing so, you risk enabling academic misconduct for which you will be held accountable. Typically, the same grade penalty is applied to the student who enabled access to their code, as to the student who used the code.

Note that if you have attempted the course in the past, the first bullet point above still applies - you must not make use of any code written for your term project when you last took the course.

Beware:

- A number of students have violated this policy in the past and have paid the price. If you find the project challenging, ask for help from a course instructor or from one of the TAs *assigned to the course this term*. Do not be tempted to copy code from someone else and do not, under any circumstances, allow another student access to your code in any way. We have a tool that identifies unusual similarities in code - it works really well and we *will* be using it.
- You must not hire a tutor to help you write or debug the code for your term project. All the code that you submit must be your own work.

General Requirements

You should commit and push your code to GitHub after completion of each task, if not more frequently. By pushing your code to GitHub you are effectively making a backup of your work which you'll be able to retrieve if anything happens to the code on your laptop.

All code must be appropriately documented:

- each class should have a class-level comment (that appears just before the class declaration) to describe the information that the class represents
- every method must be documented with requires, modifies and effects clauses, as appropriate (note that simple setter and getter methods, and test methods, do not have to be documented)

You code must adhere to our style guidelines. Note that for the term project, the maximum method length has been increased from 20 to 25 lines.

That said, there may be some cases where it does not make sense to decompose a method whose

length is greater than 25 lines. In such cases, you may include an `@SuppressWarnings("methodlength")` annotation on the line immediately preceding the method header. This instructs checkstyle not to apply the method length restriction to that particular method. You should always check with your TA before using this annotation, as any mis-use will result in a grade penalty. Note that AutoTest will list all the places in your code where the annotation is used.

Note: full marks for code coverage can be achieved only if there is at least one branch (i.e. if/else) or loop in the code included in the `model` package. In general, your term project should have sufficient complexity that you will naturally include at least one branch in the code that models your domain of interest.

It is expected that you will work on this project incrementally and will frequently push changes to GitHub. As you will be using GitHub more extensively, it's recommended that you take time to work through our GitHub tutorial.

Task 1

Make your user stories a reality!

Before you get started on this task, carefully review the Teller application that we'll be studying in more detail later in the term. Notice that the `Account` class represents an account in our banking domain and is found in the `model` package, with the corresponding test class in the `tests` folder. The `TellerApp` class provides the console based user interface and is found in the `ui` package. The application demonstrates the division between code that comprises the model and code that comprises the user interface. You need to maintain this same separation in the code that you write. Also notice that the `Main` class has no fields and that the `Main.main` method contains a single statement that creates an instance of the `TellerApp` class. Your `Main` class and `main` method should be similar, comprising no more than a few lines of code. Please keep in mind that the `Teller` application does not contain all the features necessary to obtain full marks on this phase of the term project - it serves only to illustrate how you can separate code that models your domain from code that implements the user interface.

Start by doing a domain analysis for your application. What information is changing and what is constant? Identify the different types of data that you will need to represent the information in your domain. So, for example, the to-do application will need a `Task` class to represent a single task and a `ToDoList` class to represent the tasks that have been added to the to-do list. You should then consider the methods that each of these classes must have if you are to be able to provide the functionality described in your user stories. So, if the user must be able to add a task to their to-do list, the `ToDoList` class will need an `addTask` method. The classes that you identify at this stage in your design must be added to your `model` package with the corresponding tests in the `test` folder.

The classes `MyModel` and `MyModelTest` were provided with the starter code. Either delete these classes or rename them to something more appropriate for your particular application.

Note there must be no `System.out.println` statements in the code that models your domain and no code that reads user input from the keyboard. These statements are used to present/gather information to/from the user and are therefore part of the user interface which you'll write in the next task.

When you are done, commit and push your code to GitHub.

Task 2

The next step is to add the user interface. At this stage you *must* design a console application. *Do not* attempt to design a graphical user interface. All the code for your user interface must go in the `ui` package. As user interface code is more difficult to test, in general, note that you are not required to design tests for your `ui` code on this term project. However, you must ensure that all your methods are appropriately documented.

When your application is run, a user must be able to perform each of the actions specified in your user stories. In other words, you are expected to have a working console-based application by the end of Phase 1.

When you are done, commit and push your code to GitHub.

The Lanterna Library

If you are designing a game for your term project, you might want to consider making use of the Lanterna library. It will allow you to present a basic visualization of your game. A tutorial can be found here. However, please note that use of this library is not required for Phase 1.

Be aware that in Phase 3, you will still need to switch to a GUI interface. So, you may instead want to wait until then to make something fancy so you only need to create one graphical appearance for your game.

Grading

When you are ready, commit and push your code to GitHub. Request feedback from AutoTest by making the following comment on your commit:

```
@210-bot #project
```

As you are all working on individual projects, AutoTest will obviously not give feedback on the correctness of your code. However, it will check that your code adheres to the CPSC 210 coding style and will measure the degree to which your tests cover your code. The code coverage score that you receive from AutoTest will be used, *in part*, to determine your overall grade for this phase of the term project as follows:

- 20 pts - (code coverage score * 20) - so 100% code coverage yields a score of 20, whereas 0% code coverage yields a score of 0
- 10 pts - clear and correct documentation has been provided
- 70 pts - realization of four user stories relevant to the proposed application (roughly equally weighted); to obtain any of these marks at least one of the user stories that has been realized must be of the form "add multiple Xs to a Y" as described in the Phase 0 description.

Note:

- any attempt to "fake" the code coverage score (e.g. calling a method but not using appropriate `assertEquals`, `assertTrue` and/or `assertFalse` statements to verify the expected behaviour) will result in an automatic zero for code coverage
- you must get at least 50% on this phase of the term project. If you don't achieve that grade, you will have to continue to work on this phase of the term project and request a regrade at your next lab for a maximum score of 50% on this phase of the term project
- we recommend you check the "General Requirements" section above to make sure that they have been met

During your demo for this phase of the term project, you will be asked to:

< Previous Next >

© All Rights Reserved